ComEng - Ergänzung

FS 2026 – Prof. Lars Kamm

Autoren:
Mirko Ratti

Version:
1.0.20260512
https://github.com/mirkoratti/FS2026

Inhaltsverzeichnis

1 Hex - Dez **2**

2 Instruction Set Architecture (ISA) **2**

3 Logica della ALU e Program Status Register **2**
3.1 Meccanismo delle Flag (APSR) 2

4 Componenti del Sistema (SCS) **2**
4.1 Meccanismi di Fault e Eccezioni 2
4.2 System Tick Timer (SysTick) 2

5 Gestione degli Interrupt (NVIC) **2**
5.1 Registri di Mascheramento 2

6 Stack e Gestione Memoria **2**
6.1 Estensioni e Manipolazione 2

7 Ottimizzazione e Strutture Dati **2**

8 Confronto Architetture ARM Cortex **2**
8.1 Evoluzione Cortex-M0 vs M0+ 2

9 Adressierungsarten ARM (Cortex-M0) **3**
9.1 Register Direkt, Immediate 3
9.2 Register Indirekt (ptr) 3
9.3 PC-Relative & Literal Pool 3
9.4 Registerlisten & Stack (SP-Relative) 3
9.5 Stack Framing 3

10 Il Processo di Compilazione **3**
10.1 Analisi del Compilatore 3

11 Interfacciamento I/O **3**

12 Esempi **3**

1 Hex → Dez

Hex	A	B	C	D	E	F
Dec	10	11	12	13	14	15
Bin	1010	1011	1100	1101	1110	1111

2 Instruction Set Architecture (ISA)

L'evoluzione delle architetture ARM ha portato alla coesistenza di diversi set di istruzioni.

- **ARM (32-bit) (Obsoleto):** Istruzioni a lunghezza fissa di 32 bit. Garantiscono massime prestazioni ma occupano maggior spazio in memoria.
- **Thumb (16-bit):** Sottoinsieme del set ARM con istruzioni a 16 bit. Ideale per ridurre il footprint della memoria (circa 30% di risparmio), a scapito di una minore flessibilità operativa.
- **Thumb-2 (16/32-bit):** Tecnologia a lunghezza variabile che combina istruzioni a 16 e 32 bit. Permette di ottenere una densità di codice simile a Thumb con prestazioni analoghe al set ARM a 32 bit.

3 Logica della ALU e Program Status Register

L'Unità Aritmetico-Logica (ALU) opera esclusivamente su stringhe binarie di bit, ignorando la semantica del segno (interpretazione *signed* vs *unsigned*). Tale agnosticismo è reso possibile dalla rappresentazione in complemento a due, che permette di unificare le operazioni di addizione e sottrazione.

3.1 Meccanismo delle Flag (APSR)

Al termine di ogni operazione aritmetica, la ALU aggiorna meccanicamente quattro flag fondamentali nel *Application Program Status Register* (APSR). Queste flag descrivono le proprietà del risultato binario grezzo:

- **N (Negative):** Riflette il bit più significativo (MSB) del risultato. Se $MSB = 1$, la flag è impostata a 1.
- **Z (Zero):** Impostata a 1 se tutti i bit del risultato sono pari a zero.
- **C (Carry):** Indica un riporto oltre il bit più significativo.
  - Fondamentale per operazioni *unsigned*. Invertito per la sottrazione!
  - **Meccanismo sottrazione:** $(A + \text{NOT}(B) + 1)$. Zweierkompl.
    - $C = 1 \implies A \geq B$ (Nessun prestito / Not-Borrow).
    - $C = 0 \implies A < B$ (Prestito avvenuto / Borrow).
- **V (oVerflow):** Indica un errore di segno nel calcolo in complemento a due, Fondamentale per operazioni *signed*.
  - $(+) + (+) = (-)$ Impossibile
  - $(-) + (-) = (+)$ Impossibile

3.1.1 Interpretazione del Segno e Salti Condizionati

Poiché la ALU aggiorna simultaneamente sia la flag C che la flag V, la distinzione tra variabile `int` e `uint` non risiede nell'hardware, ma nel software.

4 Componenti del Sistema (SCS)

Il **System Control Space (SCS)** gestisce le funzionalità core del processore, inclusa la configurazione del NVIC e del registro di controllo di sistema.

4.1 Meccanismi di Fault e Eccezioni

- **HardFaults:** Sempre attivati; gestiscono errori di Bus, Usage e Memory Management. Permettono una reazione rapida agli errori critici.
- **Wake-up Interrupt:** Consente la disattivazione gerarchica dei componenti per ridurre il consumo energetico (modalità Sleep/Stop).
- **Memory Protection Unit (MPU):** Genera eccezioni in caso di accessi non autorizzati alla memoria.
- **Debug Unit:** Fornisce fino a 4 breakpoint hardware per il debugging del codice.

4.2 System Tick Timer (SysTick)

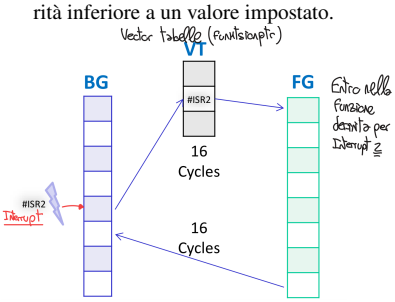
Un contatore a **24-bit** che fornisce una base dei tempi costante per il sistema operativo o i cicli di programma.

5 Gestione degli Interrupt (NVIC)

- Il **Nested Vector Interrupt Controller** gestisce la prioritizzazione e il passaggio tra il programma di background e le **Interrupt Service Routine (ISR)**.
- L'LSB in ogni vettore indica la modalità di salto (1: thumb, 0: ARM) nei Cortex-M sempre 1
- Solitamente la VT (vector table) viene slavata in memoria fra il boot code e la seconda istruzione
- Tramite VTOR (vetor table Offset Register) è possible cambaire la posizione

5.1 Registri di Mascheramento

- **PRIMASK:** 1 bit per disabilitare tutti gli interrupt (tranne HardFault e NMI).
- **FAULTMASK:** 1 bit per disabilitare interrupt e tutte le eccezioni.
- **BASEPRI:** Disabilita interrupt con priorità inferiore a un valore impostato.



	Exception Number	Vectors	CMSIS Interrupt Number	Memory Address
Interrupts	47	Interrupt#31 Vector	31	0x0000'00BC
	⋮	⋮	⋮	⋮
	18	Interrupt#2 Vector	2	0x0000'0048
	17	Interrupt#1 Vector	1	0x0000'0044
	16	Interrupt#0 Vector	0	0x0000'0040
Exceptions	15	SysTick Vector	-1	0x0000'003C
	14	PendSV Vector	-2	0x0000'0038
	13	Not Used	NA	0x0000'0034
	12	Not Used	NA	0x0000'0030
	11	SVC Vector	-5	0x0000'002C
	10	Not Used	NA	0x0000'0028
	9	Not Used	NA	0x0000'0024
	8	Not Used	NA	0x0000'0020
	7	Not Used	NA	0x0000'001C
	6	Not Used	NA	0x0000'0018
	5	Not Used	NA	0x0000'0014
	4	Not Used	NA	0x0000'0010
	3	Hard Fault Vector	-13	0x0000'000C
	2	NMI Vector	-14	0x0000'0008
	1	Reset Vector	NA	0x0000'0004
	0	MSP Initial Value	NA	0x0000'0000

Ordine di priorità

6 Stack e Gestione Memoria

Lo **Stack** è una memoria temporanea utilizzata per variabili locali e salvataggio contestuale durante le eccezioni.

- **Direzione:** Cresce verso indirizzi di memoria inferiori.
- **Stack Pointer (SP):** Punta sempre al "top of stack". È vietato leggere o scrivere al di sotto di SP.
- **Allineamento:** Normalmente allocato in step di 8 byte.
- **Velocità:** L'accesso allo stack è circa due volte più rapido del RAM/ROM pointer poiché l'indirizzo (SP) è già disponibile.

6.1 Estensioni e Manipolazione

- **SXTB/UXTH:** Estensione del segno o zero-fill (4 byte ← 1/2 byte).
- **REV/REV16:** Inversione dell'ordine dei byte (Endianness conversion).
- **CMP/CMN:** Confronti che aggiornano le flag senza salvare il risultato (es. `CMP` esegue `SUBS`).

7 Ottimizzazione e Strutture Dati

7.0.1 Switch Case

- **Pro:** Numero di cicli di clock definito e prevedibile.
- **Contro:** La condizione deve essere necessariamente una costante di tipo `int`.
- **Nota:** L'ordine dei `case` è rilevante per l'efficienza della ricerca.

7.0.2 Flusso di Controllo in Assembly

- **Selezioni (If/Else):** Si utilizza un **salto in avanti** con la **condizione invertita** rispetto al C (se la condizione falsa, salto oltre il blocco *then*).
- **Iterazioni (Loop):** Si utilizza un **salto all'indietro** alla fine del ciclo, mantenendo la **condizione assembly come in C** (salto all'inizio se la condizione è ancora vera).

7.0.3 Tecniche di Ottimizzazione

- **Loop Unrolling:** Tecnica che consiste nel ripetere il corpo del loop per ridurre l'overhead dei *branch* e degli incrementi del contatore. `#pragma unroll (n)`
  - *Vantaggio:* Maggiore velocità di esecuzione. (meno branches)
  - *Svantaggio:* Maggiore occupazione di memoria (codice più lungo).
- **Pointers (Stackless):** L'uso dei registri invece dello stack per manipolare i puntatori è significativamente più veloce (spesso più del doppio).

7.0.4 Efficienza delle Strutture Dati

- **Array Locali:** Molto **inefficienti**. alto numero di cicli solo per l'inizializzazione nello stack frame
- **Strutture:** È consigliabile **definire le strutture globalmente**. L'inizializzazione locale su stack comporta un overhead elevato di cicli di clock.
- **Bitfields:** Estremamente **inefficienti**. Sebbene risparmino memoria, richiedono complesse operazioni di *Read-Modify-Write* (LDR, Shift, Mask, OR, STR)

8 Confronto Architetture ARM Cortex

La famiglia Cortex si divide in tre profili ottimizzati per diversi scenari:
Cortex-A (Application): Calcolo generico ad alte prestazioni (Linux/Android).
Cortex-R (Real-Time): Bassa latenza e alta affidabilità (automotive/medico).
Cortex-M (Microcontroller): Ridotto consumo energetico, interrupt deterministici.

Core	Thumb	Thumb-2	HW Mult.	HW Div.	Math Sat.	DSP	FPU	Arch
Cortex-M0	Mostly	Subset	1/32 cyc	No	No	No	No	ARMv6-M
Cortex-M0+	Mostly	Subset	1/32 cyc	No	No	No	No	ARMv6-M
Cortex-M1	Mostly	Subset	3/33 cyc	No	No	No	No	ARMv6-M
Cortex-M3	Fully	Fully	1 cyc	Yes	Yes	No	No	ARMv7-M
Cortex-M4	Fully	Fully	1 cyc	Yes	Yes	Yes	Optional	ARMv7E-M
Cortex-M7	Fully	Fully	1 cyc	Yes	Yes	Yes	Optional	ARMv7E-M

8.1 Evoluzione Cortex-M0 vs M0+

- **M0+ Pipeline:** Ridotta a 2 stadi per maggiore efficienza.
- **Accesso I/O:** Supporto per Single-Cycle GPIO.
- **Debugging:** Micro Trace Buffer (MTB) opzionale per il tracciamento dei salti.
- **MPU**

8.1.1 Pipeline

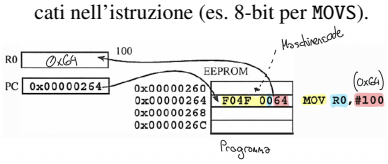
+ Es wird in jedem Cycle ein Befehl ausgeführt
- Bei sprüngen muss die Pipeline 'geleert' werden

9 Adressierungsarten ARM (Cortex-M0)

- Register direkt
- Register Indirekt
- Register Indirekt mit offset `[R?, #imm]`
- Register Indirekt mit index `[R?, R?]`

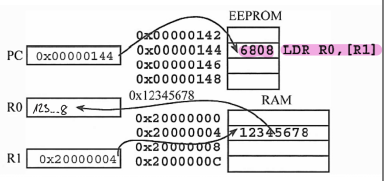
9.1 Register Direkt, Immediate

- **Immediate:** `MOV R0, #100` → Valore caricato direttamente.
- **Register:** `MOVS R2, R1` → Copia valore tra registri.
- **Limitazioni:** Gli immediati sono codificati nell'istruzione (es. 8-bit per `MOVS`).



9.2 Register Indirekt (ptr)

- **Semplice:** `LDR R0, [R1]` → $R0$ = valore all'indirizzo $R1$.
- **Con Offset:** `LDR R0, [R1, #4]` → $R0$ = valore all'indirizzo $R1+4$ (utile per array/struct).
- **Con Index:** `LDR R0, [R1, R2]` → $R0$ = valore all'indirizzo $R1+R2$.



9.3 PC-Relative & Literal Pool

- **Problema:** Istruzioni da 32-bit non possono contenere costanti da 32-bit.
- **Soluzione:** L'assembler salva la costante nel **Literal Pool** (zona memoria vicina).
- **Accesso:** `LDR R0, =0x12345678` viene tradotto in `LDR R0, [PC, #offset]`.
- Richiedono sempre **due accessi** alla memoria

9.4 Registerlisten & Stack (SP-Relative)

- **Multiple:** `LDM R7!, {R0-R3, R5}` → Carica più registri partendo da indirizzo in $R7$.
- **Stack:** `PUSH {R0-R2}`, `POP {R0-R2}`.
- **Regola d'oro:** Il registro con numero più basso (`R0`) va sempre all'indirizzo di memoria più basso. L'ordine nelle graffe non conta per l'hardware.

9.5 Stack Framing

Lo Stack Frame è lo spazio riservato nella RAM per le variabili locali di una funzione.

- **Allocazione:** `SUB SP, SP, #X` riserva X byte (lo stack cresce verso indirizzi bassi).
- **Frame Pointer** ($R7$)**:** Si usa `MOV R7, SP` per avere un riferimento fisso all'inizio del frame.
- **Accesso:** Le variabili vengono scritte/lette usando offset rispetto a $R7$, es: `STR R0, [R7, #4]`.
- **Deallocazione:** Prima di ritorno (`BX LR`), si deve pulire lo stack con `ADD SP, SP, #X`.

C-Code	Assembly

void **main**(void)
{
uint32_t a = 100;
int8_t b = -5;
 	
// user code
}
	main:
	SUB SP, SP, #8
	MOV R7, SP
	MOVS R0, #100
	MVNS R1, #4
	STR R0, [R7, #0]
	STRB R1, [R7, #4]
	; user code
	ADD SP, SP, #8
	BX LR

10 Il Processo di Compilazione

Il codice sorgente viene trasformato in binario attraverso diverse fasi:

1. **Preprocessore:** Gestisce inclusioni e macro. (`#define`, `#if`, ...)
2. **Compilatore:** Traduce il linguaggio ad alto livello in Assembly.
3. **Assembler:** Converte Assembly in codice macchina rilocabile.
4. **Linker/Binder:** Unisce file oggetto e librerie.
5. **Loader:** Converte indirizzi rilocabili in assoluti e carica in memoria.

10.1 Analisi del Compilatore

- **Analisi Lessicale:** Rimozione spazi e raggruppamento token.
- **Analisi Sintattica/Semantica:** Controllo struttura e tipi di dati.
- **Ottimizzazione:** Miglioramento del codice per velocità o spazio.
- **Generazione:** Produzione del codice target.

11 Interfacciamento I/O

Le periferiche sono mappate in memoria (*Memory Mapped I/O*) tramite registri specifici:

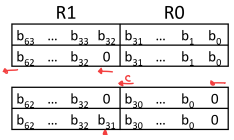
- **Datenregister:** Contiene i dati da elaborare.
- **Steuerregister:** Configurazione della periferica.
- **Statusregister:** Indica lo stato corrente dell'I/O (es. Busy, Ready).

12 Esempi

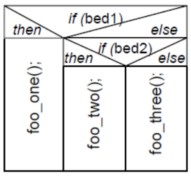
```
;Example: 64Bit Addition
LDR  r4, =addrOp ;load address of int64 array [op1,op2,res]
LDM  r4, {r0-r3} ;load op1[r1,r0] and op2[r2,r3]
ADDS r0, r0, r1  ;add lower words and set carry
ADCS r1, r2, r3  ;add higher words with carry in
ADDS r4, #8      ;set r4 to address of res
STM  r4, {r0-r1} ;store 64bit result from r0, r1 to res
```

```
; Example: 64Bit Subtraction
LDR  r4, =addrOp     ; load address of int64 array [op1,op2,res]
LDM  r4, {r0-r3}     ; load op1[r1,r0] and op2[r3,r2]
SUBS r0, r0, r2      ; sub lower words and set borrow
SBCS r1, r1, r3      ; sub higher words with borrow in
ADDS r4, #16         ; set r4 to address of res
STM  r4!, {r0-r1}    ; store 64bit result from r0, r1 to res
```

```
;Example for 64bit logical shift left
LDR  r2, =data64  ;load address of data64
LDM  r2, {r0-r1}  ;load data64 to r0, r1
LSLS r1, r1, #1   ;logical shift left r1 by 1
MOVS r3, #0       ;r3 = 0
LSLS r0, r0, #1   ;lsl r0 by 1 (carry = b31)
ADCS r1, r3       ;r1 = r1 + 0 + carry
STM  r2!, {r0-r1} ;store r0,r1 in data64
```



Equivalente a: $(\cdot 2)$

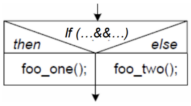


Thumb-2 Assembly

```
LDR   R0, =G2
LDR   R1, =G1
LDRH  R0, [R0]    ;R0 <- G2
SXTH  R0, R0
LDR   R1, [R1]    ;R1 <- G1
CMP   R0, R1      ;G2-G1
BGE   else_if
then:
BL    foo_one
B     endif
else_if:
CMP   R0, R1
BLE   else
BL    foo_two
B     endif
else:
BL    foo_three
endif:
```

C Source Code

```
int32_t G1; // 32Bit
int16_t G2; // 16Bit
...
if(G1 > G2)
{
  foo_one();
}
else if(G1 < G2)
{
  foo_two();
}
else
{
  foo_three();
}
```



Thumb-2 Assembly

```
LDR   R0, =G1
LDR   R0, [R0]     ;R0 <- G1
MOVS  R1, #4       ;R1 = 4
CMN   R0, R1       ;G1-(-4)
BLE   else
CMP   R0, R1       ;G1-4
BGT   else
then:
BL    foo_one
B     endif
else:
BL    foo_two
endif:
```

C Source Code

```
int32_t G1;
...
if((G1 > -4) &&
   (G1 <= 4))
{
  foo_one();
}
else
{
  foo_two();
}
```

Thumb-2 Assembly

```
0x080000F6  LDR     R1, =State
0x080000F8  LDRB    R0, [R1]      ;R0 ← State
0x080000FA  CMP     R0, #3        ;Default State R0 - 3
0x080000FC  BHI     default       ;R0 - 3 > 0
0x08000100  LSLS    R0, R0, #2    ;R0 = R0 * 4 (offset)
0x08000102  ADR     R1, lookup    ;R1 = 0x08000108
0x08000104  LDR     R0, [R1, R0]  ;R0 = val[lookup + offset]
0x08000106  MOV     PC, R0        ;PC ← Addr of case
lookup:
0x08000108  .word   0x08000130    ;addr of case 0
0x0800010C  .word   0x08000136    ;addr of case 1
0x08000110  .word   0x0800013C    ;addr of case 2
0x08000114  .word   0x08000142    ;addr of case 3
…
case_0:
0x08000130  BL      foo_one
0x08000134  B       end_switch
case_1:
0x08000136  BL      foo_two
0x0800013A  B       end_switch
case_2:
0x0800013C  BL      foo_three
0x08000140  B       end_switch
case_3:
0x08000142  BL      foo_four
0x08000146  B       end_switch
default:
0x08000148  MOVS    R0, #0
0x0800014A  STRB    R0, [R1]
end_switch:
0x0800014C  …
```

C Source Code

```
uint8_t State;      // unsigned 8Bit
…

switch(State)
{
  case 0:
    foo_one();
    break;
  case 1:
    foo_two();
    break;
  case 2:
    foo_three();
    break;
  case 3:
    foo_four();
    break;
  default:
    State = 0;
    break;
}
…
```

1. **Controllo dei limiti (Bound Check):** Verifica iniziale del valore di `State` per saltare subito al `default` se l'indice è fuori range.
2. **Calcolo dell'offset:** Moltiplicazione del valore per 4 ($State \ll 2$) tramite shift logico, poiché ogni puntatore nella look-up table occupa 32 bit (4 byte).
3. **Recupero del puntatore:** Lettura dell'indirizzo di destinazione esatto sommando l'offset calcolato all'indirizzo base della tabella.
4. **Salto (Branch):** Caricamento dell'indirizzo recuperato nel Program Counter (PC) per avviare l'esecuzione del caso specifico.